

Process Mining and Formal Verification of the Volvo IT Incident Management Process BPI Challenge 2013

Francesco Sgaramella

September 9, 2026

Abstract

This report analyses the BPI Challenge 2013 event logs from Volvo IT Belgium's VINST incident and problem management system. Three discovery algorithms are compared, model quality is assessed on four conformance dimensions, and eight temporal properties are verified against every case using finite-trace linear temporal logic. Three findings emerge. Rework dominates the process: fewer than half of all recorded events are the first occurrence of their activity within a case. The documented procedure of queueing an incident before working it is violated in the large majority of cases, a deviation established by formal verification rather than by inspection. And the Alpha miner fails on this log, defeated by the loop behaviour of the process.

1 Introduction

1.1 Context

This report applies process discovery, conformance checking and formal verification to the BPI Challenge 2013 dataset: the incident and problem management logs of Volvo IT Belgium, recorded by the VINST service management system.

1.2 Objectives

1. Derive process models from the logs using three discovery algorithms and compare them using metrics.
2. Specify properties of the incident process in temporal logic and verify them against every recorded case.
3. Quantify where time is spent and identify the structural sources of delay.
4. Determine whether the process as executed matches the process as documented.

2 Theoretical foundations

2.1 Event logs

An event log is a multiset of traces. A trace is a finite sequence of events belonging to one case, ordered by timestamp; each event carries at minimum a case identifier, an activity name and a timestamp. Writing Σ for the set of activities, a trace is an element of Σ^* and a log is a multiset over Σ^* . Two traces with the same activity sequence belong to the same variant, so few variants means a standardised process.

2.2 Petri nets

A Petri net models the control flow of a log. Different discovery techniques extract that flow from the recorded events.

2.3 Conformance dimensions

Four dimensions are used to compare the discovered Petri nets.

Fitness The extent to which the model can replay the traces in the log. A fitness of 1 means every observed case is replayable.

Precision The extent to which the model forbids behaviour that never occurred. Low precision indicates an over-permissive model.

Generalisation The extent to which the model is expected to accommodate future behaviour rather than being overfitted to the observed traces.

Simplicity A structural measure penalising the complexity of the net.

2.4 Linear temporal logic over finite traces

Properties of traces are expressed in linear temporal logic. Because a case is a finite sequence of events, the semantics used here are those of linear temporal logic interpreted over finite traces.

3 The dataset

3.1 Origin and scale

The BPI Challenge 2013 data comprises three logs recorded by Volvo IT Belgium's VINST system: one of incidents, and two of problems, divided into those open and those closed at the time of extraction.

Log	Cases	Events	Activities	From	To
closed_problems	1,487	6,660	7	2006-01-11	2012-05-31
incidents	7,554	65,533	13	2010-03-31	2012-05-22
open_problems	819	2,351	5	2006-11-07	2012-06-15

Table 1: The three Volvo IT Belgium's VINST system logs comparison

The incident log is the principal object of study; the problem logs are used for comparison in Section 7. They are not variants of one process.

3.2 The activity label is a modelling decision

Each event records a **Status** and a **Sub Status**. The two can be treated as one composite label or as independent dimensions; the composite choice increases the complexity of the resulting Petri net.

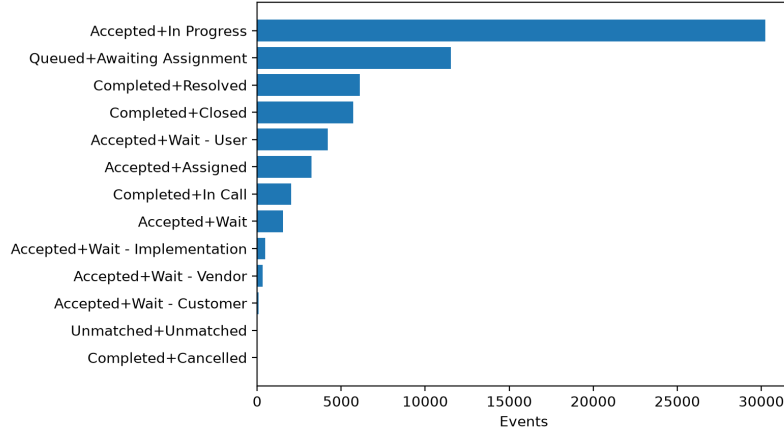


Figure 1: Event counts per activity in the incident log under the composite labelling. **Accepted+In Progress** accounts for the plurality of all events.

4 Process discovery

4.1 Algorithms

Three algorithms are compared.

Alpha miner Infers ordering relations from consecutive states and constructs a net from them. It does not handle noise and loops.

Heuristics miner Uses consecutive states frequency and applies a dependency threshold, so infrequent behaviour is filtered out.

Inductive miner Uses trees that are converted into graphs to model the net. Guarantees soundness and, at a noise threshold of zero, perfect fitness.

4.2 Results

Mode	Algorithm	Fitness	Precision	Gen.	Simpl.	Places	Trans.
status	alpha	0.119	0.400	0.734	1.000	2	4
status	heuristics	0.994	0.678	0.767	0.622	8	15
status	inductive	1.000	0.440	0.748	0.667	17	23
status+substatus	alpha	0.127	0.176	0.770	0.789	2	13
status+substatus	heuristics	0.948	0.387	0.634	0.471	27	69
status+substatus	inductive	1.000	0.234	0.783	0.634	57	78

Table 2: Discovery on the full incident log; conformance computed on a sample of cases. Both activity labellings are shown.

Three observations follow.

The fitness and precision trade off. The Inductive miner attains perfect fitness but loses precision: it buys replayability by being permissive, and it produces by far the largest net. The Heuristics miner sacrifices a little fitness for substantially better precision in a net a fraction of the size.

The Alpha miner fails. Its fitness falls below 0.15 in both labellings and the net degenerates to two places. Loops are the cause: ninety percent of cases in this log contain at least one length-one loop, which the Alpha miner provably cannot represent.

Finer labels cost precision. Moving from four activities to thirteen lowers precision for every algorithm and roughly triples model size, for the reason given in Section 3.2.

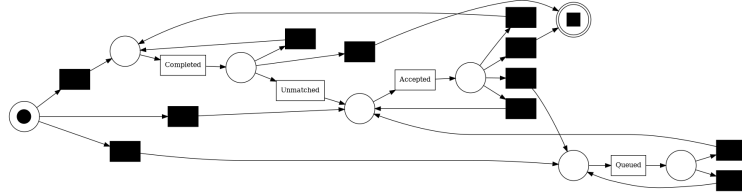


Figure 2: Heuristics miner, **Status** labelling. The best-balanced model of those compared.

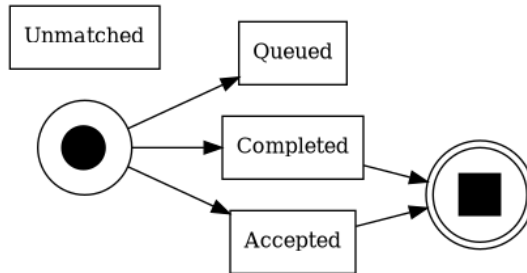


Figure 3: Alpha miner, **Status** labelling. The degenerate structure is the consequence of length-one loops, not of noise in the data.

5 Performance analysis

5.1 Method

Waiting time is measured as the interval between an event and the next event of the same case. The last event of a case has no successor and is therefore excluded.

Activity	n	Mean (h)	Median (h)	Max (h)
Completed+Closed	143	143.31	95.28	736.50
Accepted+Wait - Vendor	313	131.63	28.15	1633.67
Accepted+Wait - Implementation	491	130.22	26.00	3500.55
Accepted+Wait - Customer	101	115.45	54.39	1488.93
Completed+Resolved	6,025	114.15	176.25	192.10
Accepted+Wait - User	4,214	109.97	26.36	5059.88
Accepted+Wait	1,533	97.99	3.45	4992.31
Completed+In Call	153	70.88	1.56	677.31
Accepted+Assigned	3,220	36.31	1.36	17334.07
Queued+Awaiting Assignment	11,544	26.16	0.59	6838.52
Accepted+In Progress	30,237	10.63	0.03	7896.28
Unmatched+Unmatched	5	0.19	0.07	0.61

Table 3: Waiting time after each activity in the incident log. Both mean and median are reported; see below for why either alone misleads.

5.2 Mean and median diverge

For **Accepted+In Progress** the median waiting time is on the order of minutes while the mean is on the order of hours. The work is normally picked up immediately and occasionally stalls for a very long time. Both are reported to show the phenomenon.

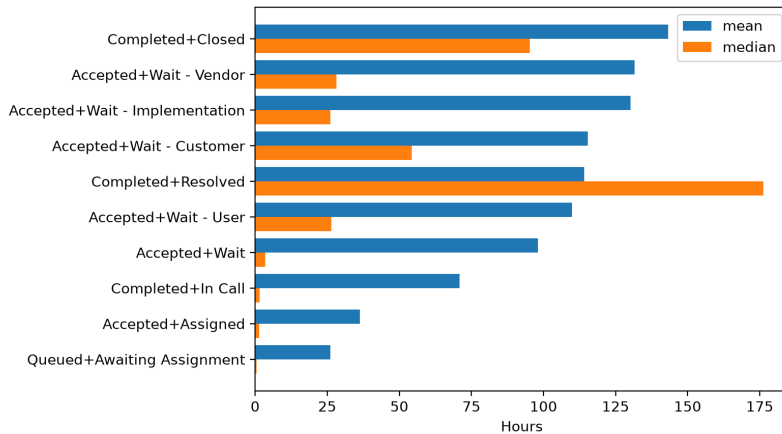


Figure 4: Mean against median waiting time for the slowest activities. Where the bars diverge sharply, the activity is usually fast and occasionally pathological.

5.3 Where the time goes

The **Wait** states dominate mean waiting time characteristic for a support process whose delays are external rather than internal, and it locates the improvement opportunity outside the service desk itself.

That **Completed+Closed** appears in Table 3 at all means a substantial number of closure events were *not* the last event of their case: work continued after the incident was formally closed. Section 6 reaches the same conclusion by another route.

5.4 Duration and variability

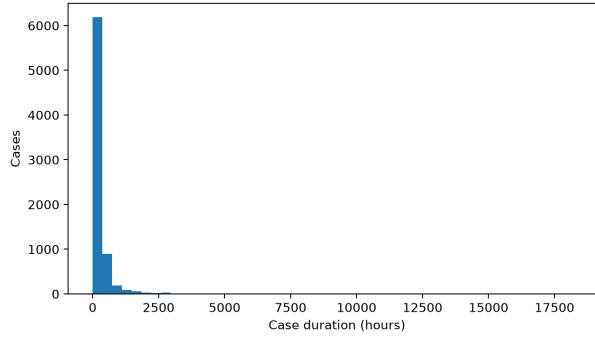


Figure 5: Distribution of case durations in the incident log.

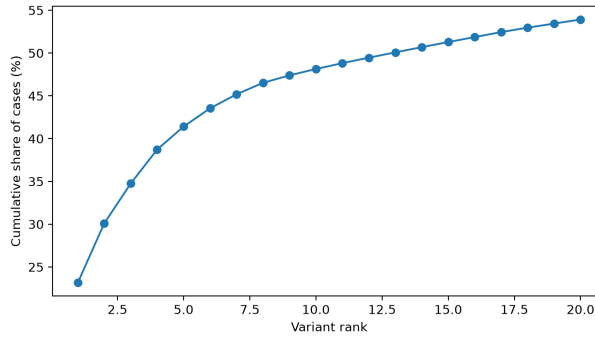


Figure 6: Cumulative share of cases covered by the most frequent variants. The curve flattens early: a large minority of cases follow paths that occur once.

The process is only weakly standardised. Distinct variants number in the thousands against seven and a half thousand cases, and the majority of variants occur exactly once. Any model of this log with high precision would be overfitted, which is the proper interpretation of the precision figures in Table 2.

6 Formal verification

6.1 Specification

Eight properties were specified in temporal logic over the composite activity alphabet, with activity labels mapped to proposition symbols (**Accepted+In Progress** becomes

accepted_in_progress). Each is evaluated independently against every case; the satisfaction ratio is the share of cases satisfying it, and violating cases are retained as counterexamples. Six properties express expectations that ought to hold; two express the documented procedure and are included because they are expected to fail, the size of the gap being the result.

Property	Satisfied	Violated	Ratio
Termination	7,546	8	0.999
Resolution	7,545	9	0.999
No work after closure	7,435	119	0.984
Queued work is picked up	7,494	60	0.992
Waiting on the user resumes	7,550	4	0.999
No unmatched events	7,549	5	0.999
Formal closure	5,591	1,963	0.740
Queueing precedes work	1,168	6,386	0.155

Table 4: Verification against all cases of the incident log.

6.2 Interpretation

The process terminates and resolves. Termination, resolution, and recovery from waiting on the user each hold for essentially every case. Whatever else is wrong with this process, cases do not disappear into it.

Work continues after closure. The property

$$G(\text{completed_closed} \rightarrow \neg F \text{accepted_in_progress})$$

fails for a small but non-trivial number of cases: an incident is formally closed and then worked again, corroborating the appearance of **Completed+Closed** in the waiting-time table of Section 5.

Formal closure is not universal. A quarter of incidents that are worked never reach **Completed+Closed**; they are settled as **Completed+In Call** instead.

Queueing does not precede work. The documented procedure is that an incident is queued for assignment and then taken up. Formally,

$$(\neg \text{accepted_in_progress}) \text{ U } \text{queued_awaiting_assignment}.$$

This holds for a small minority of cases. In the large majority, work begins without the incident ever having been queued.

6.3 A methodological result: strong next

The no-work-after-closure property was first written in the form

$$G(\text{completed_closed} \rightarrow X G \neg \text{accepted_in_progress}),$$

the natural infinite-trace formulation, which reports a satisfaction ratio of roughly one quarter, an apparently severe process failure. The figure is an artefact: **X** is false at the final position of a finite trace (Section 2.4) and most cases here *end* with **Completed+Closed**, falsifying the implication for reasons unrelated to the process. The **F**-based formulation above avoids **X** and reports the true figure of roughly 98%.

7 Aggregate assessment

To compare the three logs on one scale, a composite score aggregates four normalised components: the share of cases reaching a completed state; the share of events that are the first occurrence of their activity within their case; one minus the variant ratio; and a throughput term derived from median case duration against a fixed horizon.

Log	Score	Grade	Completion	Rework	Standardisation	Throughput
closed_problems	65.9	D	1.000	0.655	0.780	0.000
incidents	74.4	C	0.999	0.481	0.698	0.748
open_problems	70.0	D	0.435	0.746	0.778	0.942

Table 5: Composite score and its components across the three logs. Higher is better on every component.

The weighting and the throughput horizon are stipulated, so the score compares logs and labellings but carries no absolute meaning. Completion is near perfect and variability moderate, but fewer than half of all events are first occurrences of their activity: over half of the recorded work in the incident process is repetition.

8 Implementation

The analysis is a single self-contained Python package; dependencies are declared in `pyproject.toml` and resolved with `uv`. The results reported here were produced with Python 3.12, `pm4py` 2.7.23, `floater` 0.3.0, `pandas` 3.0 and `Graphviz` 2.43, with `Gradio` 6.26 for the dashboard and `Matplotlib` 3.11 for the figures. The language model clients are an optional extra.

`logs.loader.load` detects the schema, derives the activity label and returns an `EventLogBundle` carrying the `pm4py` log, the full source frame, the schema and the labelling mode. Normalisation *adds* the XES-standard columns rather than reducing each event to them, preserving the support-line, ownership and country attributes that Section 9.2 identifies as the basis of any continuation of this work. The label is computed here from the **Status** and **Sub Status** pair, under a mode that is a parameter of `load` and is then frozen into the bundle: the single point at which the modelling decision of Section 3.2 enters the system, and what allows the same log to be loaded twice under different labellings and the two models compared.

Verification encodes each case as one finite word. Since activity labels are not valid proposition symbols, `verification.ltl.Alphabet` maps them to identifiers (**Accepted+In Progress** becomes `accepted_in_progress`) and gives each position a total truth assignment, as `floater` requires. A symbol absent from the log is padded to false throughout; strict parsing rejects a symbol that names no activity anywhere in the

dataset, separating a genuinely absent activity from a mistyped one. The eight named properties are declared as data and evaluated by the same function that evaluates a formula typed into the dashboard.

The dashboard exposes the same analyses interactively across ten panels. Beyond the discovery, conformance, verification and performance results reported above, three of them are exploratory rather than confirmatory. Preprocessing composes the filters of `logs.preprocessing` in a fixed order — activities, time range, case length, duplicates, relabelling — so that a result does not depend on the order in which the controls were operated; each application replaces the session log, and reloading restores it. The anomaly panel reports the four statistical outlier families of `analysis.anomalies` — case lengths beyond a chosen number of standard deviations, activities below a share of all events, waits beyond a duration, and transitions below a support — with every threshold exposed as a control, since each is a modelling choice rather than a property of the log. The prediction panel fits the first-order Markov model of `analysis.predictor` that also produces the transition heatmap of Section 5, and reports successor probabilities and the greedy most-likely continuation from a chosen activity; the Markov assumption makes a long walk indicative rather than a forecast, and on these logs it enters the `Accepted+In Progress-Completed+Closed` cycle that Section 7 identifies as reopening.

Every function in the mining, verification and analysis packages is stateless; the only mutable state in the running dashboard is the bundle itself, held per browser session. The callbacks are therefore ordinary functions with no Gradio import, and the suite comprises 234 tests, including regression tests pinning the measured figures reported here against the real logs. All tables and figures are generated from the analysis code.

9 Conclusions

9.1 Findings

Three results stand out. Rework dominates: fewer than half of the events are the first occurrence of their activity within their case, and this drives the aggregate score down rather than any single slow step. The documented procedure is not the executed procedure (queueing before work holds for a small minority of cases and the “push to front” pattern dominates), established by formal verification over every case rather than by sampling. And the Alpha miner is inapplicable here, its failure predicted by the prevalence of length-one loops.

9.2 Threats to validity

Conformance metrics are computed on a deterministic but non-random sample, because alignment-based computation does not terminate at full scale; the headline figures use token-based replay, optimistic on unsound nets, and the implemented alignment-based computation agrees on the sample. The composite score’s weights and horizon are stipulated, so only its components are meaningful in absolute terms. `Unmatched+Unmatched` is an export artefact, retained so its prevalence stays visible, and should not be read as work.

Every finding here concerns what the system recorded: work performed but not recorded is invisible to all of these methods, and poor recording discipline appears as rework.

9.3 Further work

The loader retains organisational attributes this analysis does not yet use. The natural continuations, all supported by the data already loaded, are handover-of-work analysis across the support lines, quantification of the push-to-front pattern by organisation rather than by trace shape, and comparison of the incident and problem processes as interacting systems.